A. Evaluation Metrics

Here we will elaborate more meticulously on the setup of the evaluation.

A.1. Automatic Evaluation Metrics

EDIT Score The EDIT Score calculates how many insertions, substitutions, and deletions are needed to make two strings identical [4]. In the field of NLP, the Edit Score measures the similarity between two strings (Kim et al., 2022; Tang et al., 2023). In the task of SVG generation, we evaluate the quality of the generated SVG from the code perspective, i.e., the closer the generated SVG code is to the ground truth SVG, the higher the quality of the SVG path, indicating fewer repeated paths and missing paths. It is worth noting that we only test the direct SVG generation methods, including Iconshop and StrokeNUWA. For optimization-based methods, e.g., SD & LIVE and VectorFusion, which continuously optimize raster images and do not have a fixed ground truth SVG, it is not possible to evaluate with EDIT score. We convert all the SVGs into the code format, treat the codes as strings, and calculate the corresponding EDIT scores.

Recall Rate Based on our observations, we have noticed that for the majority of SVGs, some key paths and keywords match. For example, given the keyword "clock", the corresponding SVG should contain key SVG paths resembling clock hands. By identifying these key paths, one can roughly understand that the SVG represents a clock. Therefore, we design the Recall Score to measure how many key paths in the generated SVG can match the ground truth. Specifically, we convert both the generated SVG and the ground truth SVG into stroke tokens with VQ-Stroke and then calculate how many strokes in the generated SVG's corresponding stroke tokens appear in the ground truth using the following formula: $Recall = \frac{|P \cap \mathcal{G}|}{|\mathcal{G}|}$, where $|P \cap \mathcal{G}|$ represents the number of overlapping stroke tokens between the generated stroke tokens P and the ground truth stroke tokens $\mathcal{G}$. We convert all SVGs into the stroke token format, subsequently computing the recall rate.

A.2. Human Evaluation

Considering the potential bias and inaccuracies that automated metrics may introduce in evaluating SVGs, e.g., CLIP-Score (Kim et al., 2023), we organize a human evaluation process. In this human evaluation, we employ 10 evaluators and provide each with 100 evaluation samples. Each sample set included an SVG generated by StrokeNUWA and one generated by Iconshop. Evaluators are tasked with assessing the following aspects:

- **Prompt Alignment**: Whether the generated SVG aligns with the given prompt.
- **Overall Quality**: The general quality of the generated SVG, such as balance of content and clarity.
- **Graphic Details**: The presence of detailed elements in the generated SVG, such as intricacies outlined by curves.

Evaluators are instructed to evaluate StrokeNUWA against Iconshop in each metric, determining whether StrokeNUWA wins, ties, or loses against Iconshop. Before releasing the human evaluation cases, we conducted thorough checks to ensure there were no private information or problematic biases present in the cases. Additionally, we refrained from collecting personal information or inquiring about evaluators' private details during the annotation process.

B. Additional Ablation Experiments

B.1. Effectiveness of Different Textual Prompts

We study the effectiveness of different textual prompts on StrokeNUWA. Specifically, as shown in Fig. 10, we embed keywords into different textual prompt templates to guide SVG generation. These new prompts were then fed into the StrokeNUWA model to generate SVGs. It is worth mentioning that, for the sake of conciseness in illustrating the impact of different textual prompts, we employ the PI method as the SVG fixer and prompt the StokeNUWA with different textual prompts. We try five different textual prompts (including the one used in the main experiment). Notably, the first four templates place the keywords in different positions within the prompt, while the last one reconstructs the keywords into a natural language sentence using LLM. For instance, for the keywords "cloud" and "rain", we can rewrite them in natural language as: "Raindrops are falling from the cloud".

[4] https://en.wikipedia.org/wiki/Edit_distance

Templates for Different Textual Prompts

Template 1 (Utilize in the main experiment)
Generating SVG according to keywords: {Keywords}

Template 2 (Shuffle the Keywords order)
Generating SVG according to keywords: {Shuffled Keywords}

Template 3
Please generate an SVG that includes the following attributes: {Keywords}

Template 4
Here are some keywords: {Keywords}, generate the corresponding SVG:

Template 5 (Re-organize the keywords into natural language
{Natural language prompt including keywords}

Figure 10. Templates for different textual prompts.

Table 5. Effectiveness of Different Textual Prompts.

Prompt	Visual Performance			SVG Code Quality		
	FID (↓)	CLIPScore (↑)	HPS (↑)	Recall (↑) (Stoke Token)	EDIT (↓)	Optim / Pred Length (*Avg*)
Template 1	6.513	17.994	16.801	0.207	12,249.091	271.420
Template 2	**6.612**	**18.113**	**16.942**	**0.211**	**12,223.724**	274.006
Template 3	8.726	16.026	14.093	0.199	13,287.631	263.196
Template 4	11.607	13.852	9.097	0.101	19,167.086	181.059
Template 5	-	-	-	-	-	-

We report the StrokeNUWA performance with different textual prompts in Tab. 5. We can observe that if the textual prompt doesn't change much, i.e., it stays consistent with the training phase or has minor modifications (such as changing the order of keywords), the performance of the StrokeNUWA is not greatly affected and might even generate SVGs of higher quality (Template 2). However, if a new textual prompt is adopted, the position of the keywords becomes very important. If the keywords are still at the end of the textual prompt (Template 3), the model's performance shows a slight decrease across various metrics. However, suppose the keywords are not at the end (Template 4). In that case, StrokeNUWA fails to generate complex SVGs, resulting in failures on some complicated test samples and a significant drop in overall metrics. For the last case (Template 5), if keyword prompts are completely unused by StrokeNUWA during the training stage, i.e., the prompts during inference are entirely inconsistent with the prompts used during training, then StrokeNUWA can only generate a few simple SVGs, thus making the generated SVGs difficult to evaluate. We also provide some generated SVGs from StrokeNUWA under different textual prompts for reference in Fig. 12.

B.2. Semantic Clusters of Strokes

We adopt the following configuration of VQ-Stroke (the depth of the residual vector quantization is 2, with compression rates of 2 and 4, respectively, and a vocabulary size of 4096, which is the model configuration used in the main experiment), and performed clustering on all stroke tokens in the stroke codebook.

We manually set the number of clusters to 10 categories and analyze them using the T-SNE method[5]. We present the results of vocabulary clustering under the condition of a compression rate of 2 in Fig. 11, where we divide the clustering results into two types. Specifically, the cases in the right column of the above figure indicate that the distribution of these stroke tokens is very concentrated. These Stroke Tokens have very distinct features, such as representing Dots, Vertical Lines, or direction-specific short lines. In contrast, the cases in the left column represent strokes with less distinct features. This type of strokes includes various different categories of stroke types (for example, short/long strokes, direction-specific hooks, etc.). We believe that these types of strokes (on the left) constitute complex SVGs, and are greatly influenced by the vocabulary size set in VQ-Stroke. A larger vocabulary may learn more complex and layered strokes.

[5] https://en.wikipedia.org/wiki/T-distributed_stochastic_neighbor_embedding

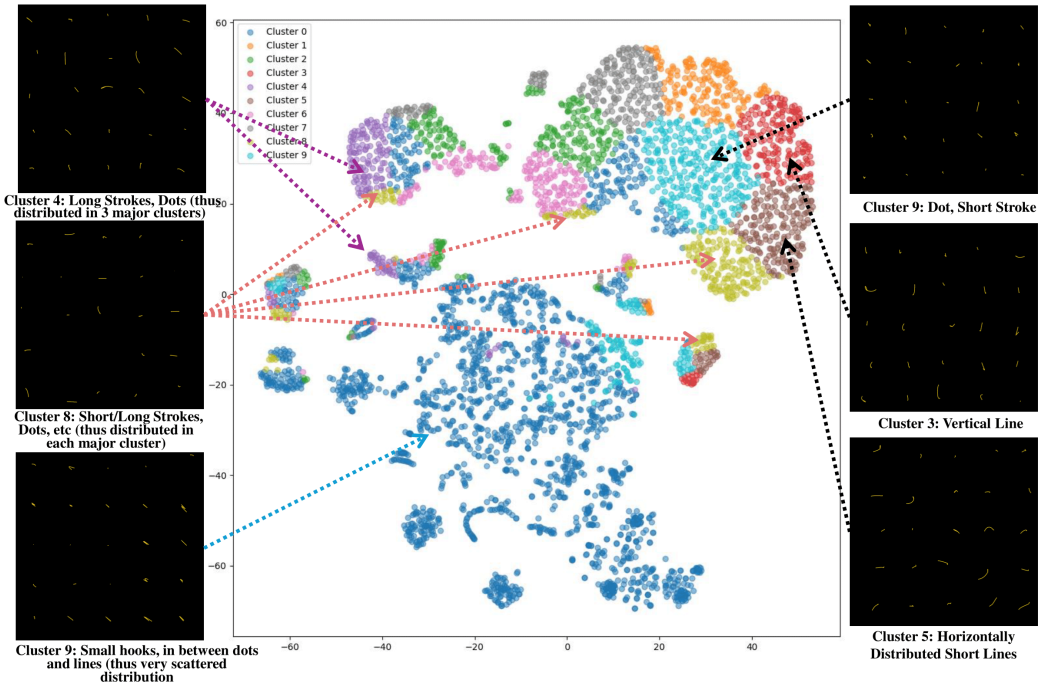


Figure 11. Semantic Clusters of Strokes

C. SVG Generation with more Attributes

In order to incorporate more elements into the process of Stroke generation, we have expanded the Code to Matrix method (as described in section 3.2.1 of the original paper). Specifically, we have added three placeholders to the original matrix, so that Equation 1: $K_{ij} = f(C_{ij}) = (T, x_0, y_0, c_0^x, c_0^y, c_1^x, c_1^y, x_1, y_1)_{ij}$ becomes $(\mathcal{K}_i^j)' = f'(\mathcal{C}_i^j) = (T, color_i, width_i, opacity_i, x_0, y_0, c_0^x, c_0^y, c_1^x, c_1^y, x_1, y_1)_i^j$. It's worth noting that the placeholders ($color_i$, $width_i$, and $opacity_i$) here can be expanded to any number, which can be defined by the user and reflected to the SVG through rules. For simplicity of demonstration and validate the feasibility of adding more attributes into "stroke tokens", we insert three special placeholders: 1) $color_i$: to control the color of the SVG path; 2) $width_i$: to control the width of the SVG path; and 3) $opacity_i$: to control the transparency of the SVG path. Since the SVG dataset we used does not provide these parameters, we randomly generated some values for a toy experiment.

For clarity, we compare the predicted strokes from VQ-Stroke model under different settings in Fig. 13, where Raw predicted SVG only uses numerical coordinates as attributes, while Random Colored SVG fills the color attribute in the SVG Path using a rule-based method. The two sub-figures in the bottom of Fig. 13 show the results trained with **VQ-Stroke** with different vocabulary sizes and attributes, i.e., color, path width, and opacity.

We retrain the EDM model in the paper using the aforementioned stroke tokens which fused with 3 attributes. Finally, we show some generated cases in Fig. 14.

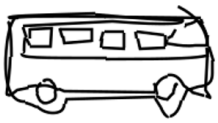
Template 1: Generate SVG according to keywords: Light Bulb, Emitting Light	**Template 3:** Please generate an SVG that includes the following attributes: Light Bulb, Emitting Light	**Template 4:** Here are some keywords: Light Bulb, Emitting Light, generate the corresponding SVG:
Template 1: Generate SVG according to keywords: Clock, Night	**Template 3:** Please generate an SVG that includes the following attributes: Clock, Night	**Template 4:** Here are some keywords: Clock, Night, generate the corresponding SVG:
Template 1: Generate SVG according to keywords: Rain, Weather	**Template 3:** Please generate an SVG that includes the following attributes: Rain, Weather	**Template 4:** Here are some keywords: Rain, Weather, generate the corresponding SVG:
		
Template 1: Generate SVG according to keywords: Car, Vehicle	**Template 3:** Please generate an SVG that includes the following attributes: Car, Vehicle	**Template 4:** Here are some keywords: Car, Vehicle, generate the corresponding SVG:

Figure 12. SVG cases generated by StrokeNUWA with different textual prompts.

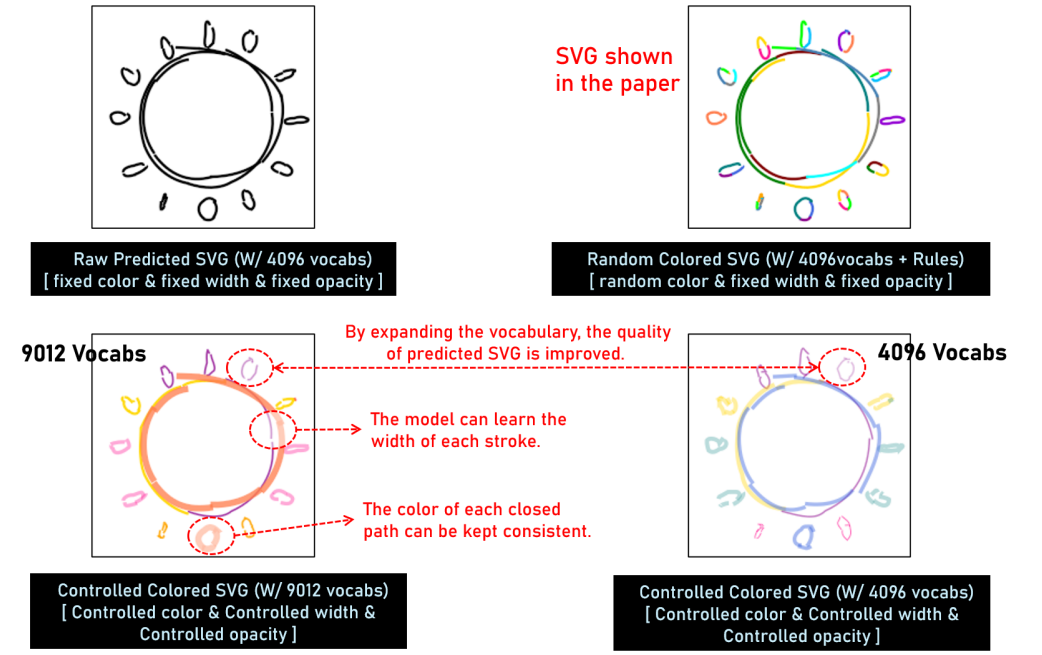


Figure 13. SVG reconstruction with more attributes.

Figure 14. SVG generated by StrokeNUWA fused with more attributes.